

VULNEX

Network Discovery • Endpoint Visibility • Vulnerability Assessment

COMPLETE PROJECT DOCUMENTATION

Current implementation: Phase 9
September 2026

Problem Statement

Organizations need a practical way to discover assets, understand endpoint inventory, assess exposed services, correlate vulnerabilities with CVEs and known exploited vulnerabilities, prioritize risk, track remediation, and produce evidence-based reports. These activities are often split across several tools, making it difficult to maintain a single operational view of network assets and endpoints.

Executive Summary — The Vulnex Solution

Vulnex is a Python and Nmap-based security assessment platform with a Streamlit dashboard that combines network discovery, full TCP-port assessment, endpoint inventory, local-device inspection, vulnerability correlation, risk prioritization, remediation tracking, historical scan comparison, and PDF reporting in one workflow.

The platform is designed around an authorized-assessment model: discover assets first, assess the selected device, collect deeper endpoint telemetry when an agent is configured, correlate technical findings with CVE and KEV information, calculate risk, track remediation, and preserve results for later comparison.

Core value
Vulnex reduces the gap between “finding an open service” and “understanding the security impact.” It connects asset identity, exposure, vulnerability context, risk, remediation state, and reporting in a single interface.

The current implementation also includes role-based access, approval-based user onboarding, persistent sessions, login rate limiting, administrator controls, persistent endpoint agents, heartbeat monitoring, on-demand inventory refresh, and automatic agent restart after system reboot.

1. Technology Stack

Vulnex uses a lightweight Python-centric architecture. The dashboard is implemented in Streamlit, Nmap performs network discovery and service assessment, endpoint collectors use operating-system APIs and Python libraries, and ReportLab produces downloadable PDF reports.

Technology / Library	Role in Vulnex	Why it is used
Python 3	Core application and agent language	Rapid development, strong security/networking ecosystem, cross-platform support.
Streamlit	Web dashboard and workflow UI	Builds an interactive security operations interface without a separate frontend framework.
Nmap	Host discovery, port scanning, service/version detection and NSE assessment	Mature network scanner with extensive protocol and vulnerability-detection capabilities.
Requests	HTTP communication	Used for Vulnex API/agent communication and external vulnerability-data retrieval.
psutil	Endpoint telemetry	Collects CPU, memory, disks, processes/services and system information across operating systems.
netifaces	Network interface information	Helps identify addresses, gateways and interface-level network details.
ReportLab	PDF report generation	Creates downloadable technical and full-device reports.
Matplotlib	Report/dashboard visualizations	Supports charts such as severity/risk distributions where required.
extra-streamlit-components	Persistent browser cookie support	Supports the persistent Vulnex login session across Streamlit refreshes.
JSON	Current lightweight persistence	Stores authentication metadata, sessions, rate-limit state, developer details and assessment data in the current Phase 9 implementation.
YAML	Structured configuration/data format	Useful for security baselines and structured policy definitions; it is not a core runtime dependency of the current Phase 9 requirements file.

System-Level Dependencies

- Nmap must be installed separately because it is an operating-system executable rather than a Python package.
- macOS commonly uses Homebrew for Nmap installation.
- Debian/Ubuntu systems can install Nmap through apt.
- Windows requires Nmap installed and available in PATH.

Design principle
Vulnex keeps the dashboard and assessment logic in Python while delegating low-level network probing to Nmap and host telemetry to platform-aware collectors.

2. Vulnex Architecture and End-to-End Flow

The current Vulnex workflow is organized into four major operational areas: Network Discovery, Endpoint Agents, Local Endpoint Inventory, and Developer/Profile administration. The vulnerability and reporting engines sit across these areas so that the same risk and remediation concepts can be applied to network and endpoint data.

Layer	Main components	Responsibilities
Presentation	Streamlit dashboard	Login, navigation, forms, cards, tables, device views, status messages and report downloads.
Authentication	Auth manager + persistent session layer	Signup, approval, login, roles, password hashing, persistent sessions, logout and rate limiting.
Assessment	Network discovery + scanner	Discover hosts, identify services, scan all TCP ports, run Nmap NSE vulnerability scripts and collect evidence.
Endpoint	Agent server + endpoint collector	Register agents, receive heartbeats, collect full host inventory and respond to refresh requests.
Local endpoint	Local collector + local scanner	Inspect the Vulnex host itself and perform local full-port assessment.
Risk	Risk/severity engine	Combine CVSS, KEV, exposure, asset criticality and other evidence into actionable severity/risk.
Remediation	Remediation engine + security store	Track Open, Acknowledged, In Progress, Resolved and Accepted Risk states.
Reporting	ReportLab/report generator	Generate assessment, executive and full-device PDF reports.
Persistence	JSON-based stores + scan history	Keep accounts, sessions, audit data, profiles, scans and remediation state available across application reruns.

End-to-End Workflow

1. Administrator or approved user signs in.
2. The user selects Network Discovery, Endpoint Agents or Local Endpoint.
3. Network Discovery identifies live assets and allows a device to be selected.
4. The selected device is assessed across TCP ports 1–65535 with service/version detection.
5. Nmap vulnerability scripts and external vulnerability data provide CVE/KEV context where available.
6. The risk engine calculates severity/risk using vulnerability and asset context.
7. The user reviews findings, evidence and remediation guidance.
8. Remediation status can be updated and preserved for later review.
9. Historical scans support comparison of newly introduced and resolved findings.
10. PDF reports provide technical and management-ready evidence.

Authorized-use requirement
Vulnex is intended for systems and networks that the operator owns or is explicitly authorized to assess. Full-port and vulnerability-script scanning can be intrusive and should be performed under an approved testing scope.

Vulnex Implementation Timeline — Phase 1 to Phase 9

The project was developed incrementally. Each phase preserved the working functionality of the previous phase while adding a defined capability. The table below keeps only the major implementation milestones and the principal issues encountered during development.

Phase	Implementation	Issue encountered	Fix / resolution
Phase 1 — Network Discovery	Converted Vulnex from a single-target scanner into a network asset-discovery workflow. Added active-network detection, Nmap discovery, device selection, per-device assessment and PDF output. Full TCP 1–65535 assessment was established.	LAN discovery could miss clients; Nmap privileges/ARP cache/AP isolation were factors.	Added ARP discovery (-PR), ICMP/TCP fallback, OS ARP-cache merge and IP deduplication. Added clearer Nmap/dependency handling.
Phase 2 — Local Inventory	Added cross-platform host inventory: OS, CPU/RAM, disks, network, software, services and users. Created the foundation later reused by endpoint agents and local-device views.	Inventory depth differs across macOS/Linux/Windows and some data requires elevated OS access.	Used platform-aware collectors and best-effort collection so unavailable fields do not break the complete profile.
Phase 3 — Dashboard Endpoint Agents	Added dashboard-managed enrollment, one-time tokens, agent IDs/secrets, authenticated heartbeats and endpoint inventory in Streamlit.	Agents initially needed a reliable enrollment lifecycle and endpoint-profile integration.	One-time token is consumed at registration; persistent agent secret is stored locally and used for subsequent heartbeats. Registered endpoints receive stale/last-seen status.
Phase 4 — Agent/Profile Integration	Expanded endpoint detail handling so agent inventory could be inspected as structured device data and prepared for merging with network assessment results.	Endpoint views initially did not expose the full inventory consistently.	Expanded hardware/network/software/service/user collection and organized endpoint views by inventory category.
Phase 5 — Reporting & UI	Added Full Device Reports combining endpoint inventory with network assessment, CVE findings and remediation guidance; refreshed the dashboard presentation.	Report generation and dashboard integration had to handle missing assessment data without failing.	Made report inputs optional/safe and kept endpoint-only reports usable when network assessment data is unavailable.
Phase 6 — Unified Dashboard	Reorganized the UI into Network Discovery, Endpoint Agents, Local Endpoint and Developer areas while retaining warnings and existing workflows.	Streamlit rendering issues occurred when conditional expressions returned UI objects and when complex values were passed directly to metrics.	Replaced inline conditional UI expressions with explicit branches and converted complex values to display-safe strings.
Phase 7 — Risk & Vulnerability Management	Added explainable 0–100 risk scoring, CVSS/KEV/exposure/asset criticality, vulnerability dashboard, asset risk, remediation lifecycle, historical comparison and executive reporting.	Dashboard/report integration produced issues such as duplicate Streamlit keys and inconsistent remediation data shapes.	Removed duplicate report rendering/key usage and normalized remediation records so both simple and metadata-bearing status values are handled.
Phase 8 — Authentication & RBAC	Added Admin/User roles, signup approval, admin controls, protected operations, developer editing, account/profile management and secure password hashing.	Authentication had to enforce permissions in application logic rather than only hiding UI controls.	Added server-side role checks, approval-state validation, salted PBKDF2-HMAC-SHA256 password hashing and administrator-only operations.
Phase 9 — Security Hardening & Agent Lifecycle	Added persistent login sessions, five-attempt/one-hour IP rate limiting, admin user deletion, endpoint re-collection, persistent background agents, reboot reconnect, local 65K scanning and report fixes.	Several integration errors appeared: refresh logout, remediation dict key error, undefined report variable, Python type-union compatibility, and foreground-only agent behavior.	Added server-side session/cookie restoration, remediation status normalization, corrected report inputs, compatible annotations, OS background services (launchd/systemd/Task Scheduler), heartbeat-driven refresh and automatic restart/reconnect.

Phase progression: Discovery → Inventory → Agents → Profile integration → Reporting/UI → Unified dashboard → Risk management → Authentication/RBAC → Persistent security operations and agent lifecycle.

3. Login, Authentication and Role-Based Access

Authentication is the entry point to the Vulnex workflow. The homepage presents User Login and User Sign Up, with a separate Admin Login area. New users are placed into a pending state until an administrator approves them.

Account Lifecycle

11. A user submits the signup form.
12. Vulnex stores the account as pending.
13. An administrator signs in and reviews pending accounts.
14. The administrator approves or rejects the account.
15. Only an approved account can complete authentication.
16. The authenticated role determines which administrative operations are available.

Password Security

- Passwords are stored as salted PBKDF2-HMAC-SHA256 hashes rather than plaintext.
- A unique random salt is used for password hashing.
- Password verification uses constant-time comparison.
- The account profile workflow supports password changes with validation.
- The intended password rule is at least 8 characters with an uppercase letter, number and symbol.

Roles

Capability	Administrator	User
Network discovery and assessment	Yes	Yes
Endpoint agent configuration	Yes	Yes
View endpoint inventory	Yes	Yes
Download reports	Yes	Yes
Change remediation status	Yes	Restricted
Approve/reject users	Yes	No
Delete endpoint agents	Yes	No
Delete user accounts	Yes	No
Edit developer details	Yes	No

Persistent Login

Streamlit session state is tied to the browser WebSocket and can be reset by a page refresh. Vulnex therefore creates a server-side session token after successful authentication, stores only its SHA-256 digest server-side, and places the session token in a persistent browser cookie. On refresh, Vulnex validates the cookie against the server-side session store and restores the authenticated account until logout or account invalidation.

Login Rate Limiting

- Wrong password attempts are tracked by client IP.
- Five wrong password attempts trigger a one-hour IP block.
- The block is persisted so a Streamlit rerun does not reset the counter.
- A successful login clears the failed-attempt counter for that IP.
- Logout revokes the server-side session and removes the browser cookie.

4. Network Discovery

Network Discovery is the first technical assessment workflow. Vulnex identifies the active local IPv4 network, discovers responsive devices, and presents the discovered inventory so the operator can select an asset for deeper assessment.

Discovery Process

17. Determine the active local IPv4 network.
18. Perform Nmap host discovery within the permitted private/loopback/link-local scope.
19. Collect IP address, MAC address where available, hostname, vendor and device-type information.
20. Display discovered assets in the Streamlit interface.
21. Allow the operator to select one device for full assessment.

Why Discovery Comes First

Discovery separates the question “what is present?” from the question “what is vulnerable?”. This creates an asset-centric workflow in which later CVE, risk and remediation findings can be associated with a specific device.

Assessment Scope

Assessment component	Current Vulnex behavior
TCP ports	All 1–65535 TCP ports using Nmap -p-
Service detection	Nmap -sV with extended version probing
Vulnerability NSE	Nmap vuln script category
OS fingerprinting	Available in extended mode when privileges permit
Ping restriction	Nmap -Pn is used where appropriate to avoid relying solely on ICMP reachability
Timeout	Extended to accommodate full-port and NSE assessment

Operational note
A 65,535-port scan plus service detection and vulnerability NSE scripts is significantly heavier than Nmap's default top-1000-port scan. Scan only approved targets and allow additional execution time.

5. Device Assessment and Evidence Collection

After selecting a discovered device, Vulnex performs a deeper assessment. The scanner builds an evidence record containing open ports, services, versions and vulnerability-script observations. This record becomes the input to CVE correlation, risk calculation and reporting.

Assessment Pipeline

Stage	Output
Target validation	Authorized target IP and assessment parameters
Port scan	Open/closed/filtered TCP ports
Service discovery	Service names and version information
Extended probing	Additional service/version evidence
NSE vulnerability scripts	Potential weakness indicators and evidence
CVE correlation	Relevant vulnerability identifiers and metadata where matched
KEV correlation	Known Exploited Vulnerability status where matched
Risk engine	Severity and combined asset/finding risk
Remediation	Recommended actions and tracked status
Report	PDF evidence and executive summary

Evidence Philosophy

- Findings should retain enough technical context to explain why they were created.
- A service/version finding is treated as evidence rather than automatically being treated as a confirmed exploitable vulnerability.
- CVE/KEV enrichment provides context but does not replace validation of the affected software and configuration.
- Reports distinguish asset information, technical findings and remediation state.

6. Endpoint Agents — Deep Host Inventory

Network scanning can identify exposed services, but it cannot provide the same depth of host inventory as a local endpoint collector. Vulnex therefore provides OS-specific endpoint agents for macOS, Linux and Windows.

Inventory Collected

Category	Examples of collected information
Hardware	CPU/model, core count, RAM, disks, system/model, architecture
Operating system	OS family, version/build, kernel, uptime and system identity
Network	Hostname, IPv4 addresses, interfaces, MAC addresses, gateway and DNS
Software	Installed applications/packages and versions where available
Services	Service names, state and configuration details where available
Users	Local users and account metadata appropriate to the OS
Updates	Available/installed update information where the platform exposes it

Dashboard View

A registered endpoint can be opened from the Endpoint Agents area. Its inventory is organized into Overview, Hardware, Software, Network, Users and Services. This allows the operator to inspect a machine without repeatedly logging into the endpoint manually.

Collect Data Again

The endpoint detail page includes a Collect Data Again action. The dashboard queues a refresh request. The agent receives that request on its next authenticated heartbeat, performs a fresh inventory collection, and sends the updated profile back to Vulnex.

Why this matters
Endpoint inventory changes over time. Software, services, users and network configuration can change after the first enrollment, so a persistent refresh mechanism is necessary for the dashboard to remain useful.

7. Persistent Endpoint Agent and Background Operation

The current agent architecture is designed so that an operator runs the OS-specific setup launcher once. After successful enrollment, the agent is installed as a persistent background service rather than requiring the shell/PowerShell script to remain open.

Operating system	Background mechanism	Boot behavior
macOS	launchd LaunchDaemon	Starts automatically after reboot and runs independently of the interactive user session.
Linux	systemd service	Enabled at boot and restarted by the service manager when configured.
Windows	Windows Task Scheduler	Configured to start automatically and operate in the background.

Heartbeat Model

- The agent sends a lightweight authenticated heartbeat approximately every 60 seconds.
- Heartbeat confirms that the endpoint is reachable and registered.
- The heartbeat can carry a request for an immediate inventory refresh.
- The agent retries communication when the Vulnex server is temporarily unavailable.
- The agent persists its assigned agent ID and secret locally after enrollment.

Inventory Refresh Model

- Normal full inventory refresh occurs approximately every five minutes.
- Collect Data Again requests an immediate refresh.
- A full collection is intentionally separated from the lightweight heartbeat to reduce unnecessary traffic and CPU usage.

Restart and Reconnect

After the endpoint reboots, the operating system starts the configured background service. The agent reads its stored identity/secret, reconnects to Vulnex and resumes heartbeat communication. The one-time enrollment token is not required for every reboot.

Stopping the Agent

- Linux: sudo systemctl stop vulnex-agent
- Linux: sudo systemctl disable --now vulnex-agent to stop and prevent automatic startup.
- macOS and Windows use their respective service/task management mechanisms.

8. Local Endpoint Inventory and Assessment

Local Endpoint is the agent-free path for inspecting the machine on which Vulnex is running. It is useful when the operator wants immediate visibility without first registering a remote endpoint agent.

Local Inventory

- Hardware and system identity
- Operating-system information
- Network interfaces and addresses
- Installed software
- Services
- Local users
- Update information where available

Local Full-Port Assessment

The local endpoint workflow also supports a full TCP-port assessment of the Vulnex host. The same core assessment approach is used: TCP ports 1–65535, service/version detection, vulnerability NSE scripts and downstream CVE/KEV/risk processing.

Local Device Report

The Full Local Device Report combines local inventory and available assessment context into a downloadable PDF. This provides a single evidence package containing the host's hardware, operating system, software, network, users, services, update information and vulnerability context.

Practical use
The local endpoint workflow is useful for validating the security posture of the Vulnex server/workstation itself, while the remote agent workflow is intended for deeper recurring visibility across managed endpoints.

9. CVE, NVD and CISA KEV Correlation

Vulnex separates raw scan evidence from vulnerability intelligence. A discovered service/version or Nmap NSE observation can be enriched with vulnerability metadata when a reliable match is available.

CVE Workflow

22. Identify the service and version from Nmap evidence.
23. Normalize the product/version information where possible.
24. Query or correlate with vulnerability information.
25. Attach matching CVE identifiers and metadata to the finding.
26. Use CVSS information as one input to risk prioritization.
27. Provide remediation context rather than treating the CVE identifier alone as proof of exploitation.

KEV Workflow

The CISA Known Exploited Vulnerabilities catalog is used as an additional prioritization signal. A vulnerability appearing in KEV indicates that it has been identified by CISA as known to be exploited in the wild. Vulnex uses KEV status as an input to its risk model and dashboard.

Signal	Meaning in Vulnex
CVE	Standard identifier for a publicly catalogued vulnerability.
CVSS	Severity/rating information used as one risk input.
KEV	Additional prioritization signal for vulnerabilities known to be exploited.
NSE evidence	Scanner-level evidence that may indicate a vulnerability condition.
Service exposure	Context showing whether the affected service is externally/reachable within the assessed network.

Important distinction
CVE, CVSS and KEV are enrichment and prioritization signals. They should be interpreted alongside the actual affected asset, version, configuration and scan evidence.

10. Risk and Severity Engine

A vulnerability score alone does not fully describe operational risk. Vulnex therefore combines vulnerability intelligence with asset and exposure context to create a more useful risk view.

Risk Inputs

- CVSS severity or score where available
- KEV status
- Exposed-service information
- Asset criticality
- Number and type of affected assets
- Vulnerability count and severity distribution
- Endpoint security posture and inventory context where available

Risk Output

Output	Purpose
Critical	Immediate attention for findings with the highest combined impact/exposure signals.
High	High-priority findings requiring timely remediation.
Medium	Meaningful risk requiring planned remediation.
Low	Lower-priority findings or lower-impact exposure.
Unknown	Insufficient data to confidently assign a severity.

Asset Risk View

- Per-device risk score
- Vulnerability count
- Exposed services
- Outdated software indicators
- Endpoint posture
- Historical risk movement where previous scans exist

Decision support
The risk engine is designed to organize technical evidence for operational prioritization. It should not be treated as a replacement for analyst validation, business impact assessment or an organization's formal risk-acceptance process.

11. Remediation Tracking

Finding a vulnerability is only one part of security operations. Vulnex stores remediation state so teams can distinguish newly discovered issues from issues that are being handled or have already been resolved.

Status	Meaning
Open	Finding requires action and has not yet entered a tracked remediation stage.
Acknowledged	Finding has been reviewed and accepted into the remediation workflow.
In Progress	Remediation work has started.
Resolved	The issue has been remediated or otherwise verified as no longer present.
Accepted Risk	The organization has formally chosen to accept the residual risk.

Remediation Lifecycle

28. Finding is created from network/endpoint assessment.
29. Risk and severity context is attached.
30. Analyst reviews evidence and remediation guidance.
31. Status is updated as work progresses.
32. Later scans can be compared against the historical record.
33. Resolved or accepted findings can be tracked separately from open issues.

Data Consistency

The remediation store is designed to tolerate both simple legacy status strings and newer status records containing additional metadata. This prevents dashboard errors when older assessment data is displayed alongside newer remediation records.

12. Historical Scanning and Change Detection

Security posture is not static. Vulnex retains scan history so the operator can compare the current state with earlier assessments.

Historical Questions Vulnex Can Support

- Which vulnerabilities are newly introduced?
- Which vulnerabilities have been resolved?
- Did an asset's risk score change?
- Did the number of exposed services increase or decrease?
- Did the software/service inventory change?
- What remediation statuses remain open?

Comparison Model

Comparison	Example result
Current vs previous findings	New CVE introduced on a previously clean device.
Current vs previous risk	Asset risk moved from 5.2 to 7.5.
Open ports	A new network service became reachable.
Resolved findings	Previously reported vulnerability no longer appears.
Inventory	New software package or service appeared.

Operational value
Historical comparison changes Vulnex from a one-time scanner into a posture-monitoring workflow. It supports change awareness rather than only point-in-time reporting.

13. Reporting and Evidence

Vulnex produces downloadable PDF reports so assessment results can be shared, archived and reviewed outside the Streamlit dashboard.

Report Types

Report	Typical content
Device assessment report	Target identity, open ports, services, vulnerabilities, CVE/KEV context, risk and remediation.
Full endpoint/device report	Hardware, OS, network, software, services, users, updates and security assessment context.
Local device report	Local host inventory plus available full-port and vulnerability assessment context.
Executive risk report	Management-level summary of assets, vulnerabilities, severity, risk and remediation posture.

Evidence Structure

- Asset identity
- Assessment timestamp
- Port/service evidence
- Vulnerability identifiers and severity
- KEV indicators
- Risk score/context
- Remediation recommendation
- Remediation status
- Before/after or historical comparison where available

Why PDF Reporting Matters

A report provides a stable evidence artifact for technical review, management communication, remediation tracking and project demonstrations. It also reduces the dependency on keeping the dashboard open to understand what was found at the time of assessment.

14. Dashboard and User Experience

The Streamlit interface is designed as a security operations dashboard rather than a collection of unrelated scripts. The landing page uses large operational cards and the project/developer information is intentionally placed at the end of the homepage.

Homepage area	Primary purpose
Network Discovery	Discover and assess network devices.
Endpoint Agents	Register, monitor and inspect managed endpoints.
Local Endpoint	Inspect and assess the local machine.
Developer Details	Project and developer information.
My Account Profile	View account details and manage credentials.

Endpoint Detail Navigation

Endpoint pages are organized into tabs/sections for Overview, Hardware, Software, Network, Users and Services. This avoids placing all telemetry into one long page and allows an analyst to focus on the category relevant to the current investigation.

Security Operations Dashboard

- Total vulnerabilities
- Critical/high findings
- Affected assets
- Open ports
- KEV vulnerabilities
- Average/combined risk
- Severity distribution
- Remediation status
- Asset risk view

15. Security Controls and Hardening

Vulnex itself handles sensitive security information, so the platform includes controls intended to reduce unauthorized access and unsafe operations.

Control	Current implementation
Authentication	Account-based login with approval workflow.
Password storage	Salted PBKDF2-HMAC-SHA256 hashes.
RBAC	Administrator and User roles.
Persistent sessions	Server-side session records plus browser cookie.
Rate limiting	Five wrong password attempts per IP followed by one-hour block.
Audit trail	Authentication and administrative events are logged.
Agent enrollment	One-time enrollment token and persistent agent credentials.
Agent heartbeat	Authenticated periodic heartbeat.
Request validation	Validation around agent/API interactions.
Admin deletion controls	User and endpoint deletion restricted to administrators.
Authorized-use warnings	Assessment UI reminds operators to scan only authorized systems.

Deployment Hardening Recommendations

- Use HTTPS/TLS in any deployment carrying credentials or security telemetry.
- Protect the JSON/authentication directory with appropriate filesystem permissions.
- Place Vulnex behind a trusted reverse proxy when exposed beyond a local environment.
- Only trust X-Forwarded-For/X-Real-IP headers from a controlled proxy.
- Back up scan history and reports securely.
- Use centralized identity management and a stronger production database for multi-tenant deployments.
- Do not expose agent enrollment tokens or secrets in public repositories.

16. Data Storage and Project Structure

The current Phase 9 implementation uses a lightweight file-backed persistence approach. This is practical for a project/demo deployment and keeps the application portable, while leaving a clear path toward a centralized database in a production version.

Location / component	Purpose
scans/auth/users.json	Account metadata and authentication records.
scans/auth/sessions.json	Server-side persistent session records.
scans/auth/login_rate_limits.json	Failed-login counters and temporary IP blocks.
scans/auth/auth_audit.log	Authentication and administrative audit events.
scans/auth/developer.json	Editable developer/project profile data.
scans/profiles/	Endpoint/device profile storage.
reports/	Generated PDF reports.
discovery/	Network discovery logic.
agent/	Endpoint agent and collector.
agent_server.py	Agent registration, heartbeat and collection request handling.
scanner.py	Network/port/service assessment logic.
risk_engine.py	Risk/severity calculation.
cve_fetcher.py	CVE enrichment.
remediation_engine.py	Remediation recommendations and workflow.
security_store.py	Remediation state and related security records.

Why File-Backed Storage Was Used

- Simple local deployment.
- No separate database server is required.
- Easy demonstration and portability.
- Human-readable operational data where appropriate.

Future architecture
For larger deployments, migrate authentication, sessions, scan history, remediation state and asset inventory into a transactional database with encrypted secrets and role-aware access.

17. Deployment and Operating-System Commands

Vulnex can be launched with Python/Streamlit, while Nmap must be installed as a system dependency. Endpoint agents use OS-specific setup launchers.

Run Vulnex

Platform	Example
macOS/Linux	python3 app.py
Any supported environment	python3 -m streamlit run app.py

Endpoint Agent Setup

OS	Launcher	Typical command
macOS	vulnex_agent_<name>_macos.sh	chmod +x vulnex_agent_<name>_macos.sh && sudo ./vulnex_agent_<name>_macos.sh
Linux	vulnex_agent_<name>_linux.sh	chmod +x vulnex_agent_<name>_linux.sh && sudo ./vulnex_agent_<name>_linux.sh
Windows	vulnex_agent_<name>_windows.ps1	PowerShell as Administrator; Set-ExecutionPolicy -Scope Process Bypass; .\vulnex_agent_<name>_windows.ps1

Linux Agent Service Operations

- Status: sudo systemctl status vulnex-agent
- Start: sudo systemctl start vulnex-agent
- Stop: sudo systemctl stop vulnex-agent
- Restart: sudo systemctl restart vulnex-agent
- Disable at boot: sudo systemctl disable --now vulnex-agent
- Live logs: sudo journalctl -u vulnex-agent -f

Important

The endpoint launcher is intended to be run once with the required administrative privileges. After installation, the background service manages the agent process and starts it after reboot.

18. Testing, Validation and Current Limitations

The project has been iteratively tested while new functionality was added. The current phase includes syntax/compatibility validation and focused authentication/agent checks.

Validated Areas

Area	Validation
Python compatibility	Project modules compiled/validated after resolving modern type-annotation compatibility issues.
Authentication	Login, persistent session, rate-limit and role behavior tested.
Remediation	Legacy/simple status and metadata-bearing status records handled.
Local reporting	Full local-device report generation tested after fixing report-generation errors.
Full-port scanning	Network and local assessment paths configured for TCP 1–65535.
Endpoint refresh	Collect Data Again workflow queues inventory refresh through heartbeat.
Background agent	OS-specific persistent service approach implemented.
ZIP/package integrity	Release packages validated before delivery.

Known Limitations

- A full 65,535-port scan plus vulnerability NSE can be slow and may produce false positives or incomplete results depending on network conditions.
- Nmap vulnerability scripts are indicators and should be validated against the actual product/version/configuration.
- Local and endpoint inventory depth depends on operating-system permissions and available APIs.
- JSON storage is appropriate for a project deployment but is not ideal for high-concurrency multi-tenant production use.
- Persistent sessions require secure deployment practices and should be protected by HTTPS in real environments.
- Agent communication requires the Vulnex server to be reachable; retry behavior helps with temporary outages but does not eliminate network dependency.

Recommended Next Improvements

- Move persistent operational data to PostgreSQL/SQLite depending on deployment scale.
- Add centralized identity/OIDC or SSO.
- Add scheduled assessments and alerting.
- Add ticketing integrations and remediation ownership.
- Add agent health/last-seen dashboards and version management.
- Add asset tagging, CSV/JSON export and API integrations.
- Add network topology visualization and attack-surface views.

19. Author and Project Contribution

Amar Kant Yadav

Cybersecurity-focused developer • Vulnerability Assessment • Security Automation

About the Author

Amar Kant Yadav is an aspiring cybersecurity analyst with a practical interest in both offensive security and defensive security. His focus is on understanding how attacks work and translating that understanding into detection, analysis and defensive controls.

He is pursuing a BTech in Computer Science and has hands-on exposure to penetration testing, vulnerability assessment, threat detection, security automation, networking and system security. His technical toolkit includes Python, C/C++, DSA/OOP concepts, Kali Linux, Nmap, Wireshark and Splunk.

His project work includes vulnerability scanners, intrusion-detection/security systems, malware-analysis tooling and security-focused automation. Vulnex represents this combined approach by connecting reconnaissance, vulnerability assessment, endpoint visibility, risk prioritization and remediation reporting in one platform.

Project Focus

- Network asset discovery and assessment
- Endpoint visibility and recurring inventory
- CVE/KEV-based vulnerability context
- Risk and remediation management
- Security automation and reporting

Conclusion

Vulnex has evolved from a network scanner into a broader security assessment platform. Its current workflow connects discovery, deep assessment, endpoint inventory, vulnerability intelligence, risk scoring, remediation tracking, historical comparison, reporting and access control. The architecture is intentionally modular so that future versions can replace file-backed storage with a production database, add centralized identity, introduce scheduled monitoring and integrate Vulnex into larger security operations workflows.

Final project objective
Provide a practical, unified and explainable security-assessment workflow that helps an authorized operator understand what assets exist, what is exposed, what vulnerabilities matter, how risk changes, and what remediation work remains.